

# JUIN 2025

# COMPTE

# RENDU FINAL

LUCCI



## Sommaires

|                                 |           |
|---------------------------------|-----------|
| <b>Introduction :</b> .....     | <b>2</b>  |
| <b>ADS-B :</b> .....            | <b>3</b>  |
| <b>Trame 17 :</b> .....         | <b>3</b>  |
| <b>Python :</b> .....           | <b>5</b>  |
| <b>GNU radio :</b> .....        | <b>8</b>  |
| <b>Mesure d'antenne :</b> ..... | <b>13</b> |
| <b>Conclusion :</b> .....       | <b>17</b> |

## Introduction :

Ce rapport présente l'analyse et la surveillance des aéronefs en temps réel à l'aide du système ADS-B, un système permettant la transmission automatique des informations de position des avions grâce aux récepteurs GPS embarqués. Ces informations sont reçues par des stations au sol ou d'autres aéronefs offrant ainsi un suivi précis. L'étude inclut également l'utilisation de GNU Radio, Python et des outils de mesure d'antenne pour capturer et analyser les trames ADS-B. Enfin des mesures d'adaptation d'impédance de l'antenne et de SWR sont effectuées avec une analyse des résultats obtenus.

## ADS-B :

Le ADS-B (Automatic Dependent Surveillance–Broadcast) est un système de surveillance utilisé dans l'aviation pour la transmission automatique des informations de position d'un aéronef. Ce système repose sur l'utilisation de récepteurs GPS à bord de l'avion qui permettent de calculer sa position, sa vitesse et d'autres données pertinentes et de les diffuser en temps réel via un signal radio.

Le signal ADS-B est reçu par des stations au sol ou d'autres aéronefs permettant ainsi de suivre la position de l'avion sans nécessiter de radar. Ce système permet une surveillance plus précise et plus continue des aéronefs tout en réduisant les coûts et les besoins en infrastructures par rapport aux systèmes radar traditionnels.

L'ADS-B est particulièrement utilisé dans les zones où le radar n'est pas disponible, comme dans les espaces aériens éloignés ou en mer. Les données transmises par ADS-B incluent des informations telles que l'ICAO, la position géographique (latitude, longitude), l'altitude, ainsi que des informations sur la vitesse et le cap. Ce système contribue ainsi à améliorer la sécurité aérienne et la gestion du trafic aérien.

## Trame 17 :

La trame 17 dans le contexte du système ADS-B est un type spécifique de message transmis par un avion qui contient des informations cruciales comme dans notre cas l'ICAO. Permettant ainsi d'identifier chaque avion de manière unique au niveau mondial.

Il convient de se rendre sur le site « <http://jasonplayne.com:8080/#> ». Ensuite il faut se rendre dans la partie DF17 puis dans la catégorie 04/03 pour l'ICAO qui était utilisé.



Figure 1 = DF17

DF17 qui est donc la trame 17 utilisé pour l'ICAO et flightNumber.



|                      |     |                                                           |       |                                      |        |        |        |        |        |        |        |        |                  |    |
|----------------------|-----|-----------------------------------------------------------|-------|--------------------------------------|--------|--------|--------|--------|--------|--------|--------|--------|------------------|----|
| MODE S Packet:       |     |                                                           |       |                                      |        |        |        |        |        |        |        |        |                  |    |
| Length               |     | : 56 bits                                                 |       |                                      |        |        |        |        |        |        |        |        |                  |    |
| Frame                |     | : 8D7C80AD2358F6B1E35C60FF1925                            |       |                                      |        |        |        |        |        |        |        |        |                  |    |
| DF: Downlink Format  |     | : (17) ADS-B                                              |       |                                      |        |        |        |        |        |        |        |        |                  |    |
| CA: Plane Mode S Cap |     | : (5) Level 2,3 or 4. can set code 7. is airborne         |       |                                      |        |        |        |        |        |        |        |        |                  |    |
| VS: Vertical status  |     | : Airborne                                                |       |                                      |        |        |        |        |        |        |        |        |                  |    |
| AA: ICAO             |     | : 7C80AD                                                  |       |                                      |        |        |        |        |        |        |        |        |                  |    |
| Registration         |     | : VH-ZPB                                                  |       |                                      |        |        |        |        |        |        |        |        |                  |    |
| ME : ADSB Msg Type   |     | : (4) Aircraft Identification and Category                |       |                                      |        |        |        |        |        |        |        |        |                  |    |
| CAT: Aircraft Cat    |     | : (0:3) Large (75000 to 300000 lbs)                       |       |                                      |        |        |        |        |        |        |        |        |                  |    |
| flight Number        |     | : VOZ1851                                                 |       |                                      |        |        |        |        |        |        |        |        |                  |    |
| Wake Type            |     | : (TC:4 CAT:3) - Medium 2 (between 34000 kg to 136000 kg) |       |                                      |        |        |        |        |        |        |        |        |                  |    |
| DF                   | CA  | AA                                                        | TC    |                                      | CAT    | CHAR   | CHAR   | CHAR   | CHAR   | CHAR   | CHAR   | CHAR   | CHAR             | PI |
| 10001                | 101 | 011111001000000010101101                                  | 00100 | 011                                  | 010110 | 001111 | 011010 | 110001 | 111000 | 110101 | 110001 | 100000 | 1111111100011001 |    |
| 0                    | 5   | 8                                                         | 32    | 37                                   | 40     | 46     | 52     | 58     | 64     | 70     | 76     | 82     | 88               |    |
| DF                   |     | (0-4) 5 bits                                              |       | Downlink Format: downlink descriptor |        |        |        |        |        |        |        |        |                  |    |

Figure 2 = trame

AA qui est l'ICAO, en regroupant par groupe de 4 bits et en convertissant en Hexadécimal, il est possible de retrouver l'ICAO.

Dans ce cas :

0111 = 7, 1100 = C, 1000 = 8, 0000 = 0, 1010 = A, 1101 = D

Donc l'ICAO est 7C80AD

Le flight number est dans les CHAR qui sont codés en ASCII UTF8. S'il est compris 1 et 26 en décimal il faut ajouter rajoute un 1 devant pour ajouter 64 en décimal et donc obtenir une lettre en ASCII. S'il est compris entre 48 à 57 il s'agit d'un chiffre, il ne faut donc pas rajouter de 1 devant.

1010110 = 86 en décimal est donc « V » en ASCII

1001111 = 79 en décimal → « O » en ASCII

1011010 = 90 en décimal → « Z » en ASCII

110001 = 49 en décimal → « 1 » en ASCII

111000 = 56 en décimal → « 8 » en ASCII

110101 = 53 en décimal → « 5 » en ASCII

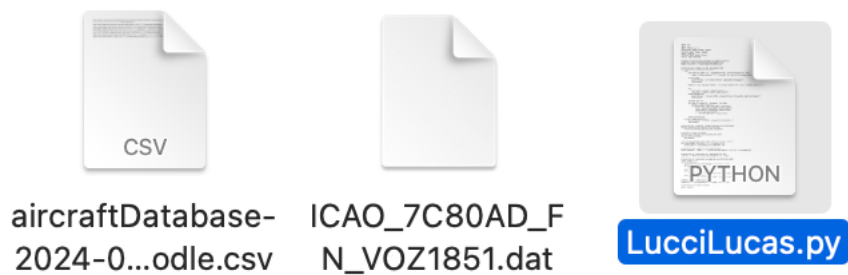
110001 = 49 en décimal → « 1 » en ASCII

Donc flight number est VOZ1851.

Donc la tram 17 permet de retrouver ainsi le flight number et l'ICAO, qui est appliqué par la suite dans le programme python.

## Python :

Le programme va avoir pour but de récupérer les données d'un avion depuis une trame .dat, l'icao ainsi que le flight number de l'avion. Puis à partir de l'icao et du flight number dans une database d'avion en .csv, le programme récupère la registration de l'avion, permettant par la suite à partir celui-ci de récupérer l'image de l'avion sur flight radar. Toutes ces données de l'avion sont affichées sur une page externe faite à partir de tkinter.



*Figure 3 = dossier programme*

La database et la trame sont dans le même dossier avec le programme permettant la bonne lecture des dossiers dans le programme.

```
1 import sys
2 import csv
3 import tkinter as tk
4 from tkinter import Label, Canvas
5 from PIL import Image, ImageTk
6 import requests
7 from bs4 import BeautifulSoup
8 from io import BytesIO
9
10 # Chemin du fichier CSV contenant les immatriculations
11 CSV_PATH = "aircraftDatabase-2024-01_moodle.csv"
12 BINARY_FILE_PATH = "ICAO_7C80AD_FN_VOZ1851.dat"
13
```

*Figure 4 = Bibliothèques et lecture fichiers*

Dans un premier temps les bibliothèques ainsi que la lecture des fichiers sont faites.

```

56 # Extraction dynamique de l'ICAO depuis un fichier binaire
57 with open(BINARY_FILE_PATH, 'rb') as binary_file:
58     binary_data = list(binary_file.read(107 * 2))
59
60 # Conversion des octets en une chaîne binaire
61 binary_message = "10001" + "".join(str(binary_data[i * 2]) for i in range(107))
62
63 # Extraction et conversion de l'ICAO (bits 8 à 32)
64 icao_hex = format(int(binary_message[8:32], 2), '06X')
65
66 # Extraction et conversion du numéro de vol (bits 40 à 88)
67 flight_number = ""
68 for i in range(8):
69     char_bits = binary_message[40 + i * 6: 46 + i * 6]
70     if len(char_bits) < 6:
71         break
72     char_value = int(char_bits, 2)
73     if 1 <= char_value <= 26:
74         flight_number += chr(char_value + 64) # Lettres A-Z
75     elif 48 <= char_value <= 57:
76         flight_number += chr(char_value) # Chiffres 0-9

```

Figure 5 = Extraction ICAO

Ce programme permet dans un premier temps l'extraction de l'icao dans le fichier, ensuite de faire la conversion des octets en une chaîne de caractère puis l'extraction et la conversion de l'icao qui est du bit 8 à 32.

Dans le même fichier le flight number est également extrais du bit 40 à 88. Puis par la suite la conversion est faite permettant de remettre en chaîne de caractère ASCII et de ainsi trouvé le flight number.

```

14 # Fonction pour charger la base de données ICAO
15 def load_aircraft_database(csv_path):
16     try:
17         with open(csv_path, "r", encoding="utf-8", errors="replace") as file:
18             lines = [line.replace('""', '').strip() for line in file.readlines()]
19
20         if not lines:
21             print("Erreur : Le fichier CSV est vide après nettoyage.")
22             return None
23
24         headers = [col.replace("\u00ff", "").strip().lower() for col in lines[0].split(",")]
25
26         try:
27             icao_index = headers.index("icao24")
28             reg_index = headers.index("registration")
29         except ValueError:
30             print("Erreur : Colonnes ICAO ou Registration introuvables après nettoyage.")
31             return None
32
33         aircraft_db = {}
34         for line in lines[1:]: # Ignorer l'en-tête
35             values = line.split(",")
36             if len(values) > max(icao_index, reg_index):
37                 icao = values[icao_index].strip().upper()
38                 registration = values[reg_index].strip()
39                 if icao and registration:
40                     aircraft_db[icao] = registration
41
42         return aircraft_db
43     except FileNotFoundError:
44         print(f"Erreur : Fichier {csv_path} introuvable.")
45         return None

```

Figure 6 = fichier CSV

Ce programme charge le fichier .csv, le nettoie par la suite, ensuite une fois nettoyée il extrait le numéro de registration de l'avion à partir de l'icao dans le fichier et retourne un dictionnaire contenant ces données. Si le fichier est mal formaté ou introuvable, un message d'erreur est affiché.

La recherche de l'immatriculation est faite ci-dessous.

```
78 # Recherche de l'immatriculation
79 registration = get_registration(icao_hex, aircraft_db)
```

Figure 7 = Registration

```
105 # Récupération de l'image JetPhotos depuis la page Flightradar24
106 try:
107     headers = {
108         "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)"
109     }
110     response = requests.get(flihttradar_url, headers=headers, timeout=10)
111     soup = BeautifulSoup(response.text, "html.parser")
112
113     # Recherche de l'image JetPhotos
114     img_url = None
115     for img in soup.find_all("img"):
116         src = img.get("src", "")
117         if "cdn.jetphotos.com" in src:
118             img_url = src
119             break
120
121     if img_url:
122         print("Image JetPhotos trouvée :", img_url)
123         # Télécharger et afficher l'image
124         response = requests.get(img_url, stream=True)
125         response.raise_for_status()
126         img_data = Image.open(BytesIO(response.content))
127         img_data.thumbnail((500, 300))
128         jet_img = ImageTk.PhotoImage(img_data)
129         jet_img_label = Label(root, image=jet_img, bg="#2C3E50")
130         jet_img_label.image = jet_img
131         jet_img_label.pack(pady=10)
132     else:
133         Label(root, text="Image de l'avion non trouvée", font=("Arial", 12, "italic"), fg="orange", bg="#2C3E50").pack(pady=10)
134 except Exception as e:
135     print("Erreur lors du scraping de l'image JetPhotos :", e)
136     Label(root, text="Erreur lors du chargement de l'image", font=("Arial", 12, "italic"), fg="red", bg="#2C3E50").pack(pady=10)
```

Figure 8 = Récupération Image

Ce programme permet de récupérer et d'afficher l'image de l'avion en récupérant l'URL hébergée sur JetPhoto via le site FlightRadar à partir de la registration de l'avion.

```
80 # Création de la fenêtre Tkinter
81 root = tk.Tk()
82 root.title("Informations de Vol")
83 root.geometry("600x700")
84 root.configure(bg="#2C3E50") # Couleur de fond sombre
85
86 # Titre
87 Label(root, text="Informations de Vol", font=("Arial", 18, "bold"), fg="white", bg="#2C3E50").pack(pady=10)
88
89 # Affichage des informations
90 Label(root, text=f"ICAO: {icao_hex}", font=("Arial", 14), fg="white", bg="#34495E").pack(pady=5)
91 Label(root, text=f"Flight Number: {flight_number.strip()}", font=("Arial", 14), fg="white", bg="#34495E").pack(pady=5)
92 Label(root, text=f"Registration: {registration}", font=("Arial", 14, "bold"), fg="white", bg="#34495E").pack(pady=5)
93
94 # Ajouter le lien Flightradar24
95 import webbrowser
96 flightradar_url = f"https://www.flightradar24.com/data/aircraft/{registration}"
97
98 def open_flightradar(event):
99     webbrowser.open(flihttradar_url)
100
101 link_label = Label(root, text="Voir sur Flightradar24", font=("Arial", 12, "underline"), fg="blue", bg="#2C3E50", cursor="hand2")
102 link_label.pack(pady=10)
103 link_label.bind("<Button-1>", open_flightradar)
```

Figure 9 = Tkinter

La page Tkinter permettant d'afficher les différentes informations de l'avion récupérer ainsi que son image et le lien vers flightRadar sont fait par le programme ci-dessus.

Beaucoup d'information son purement esthétique permettant de rendre la page plus présentable avec des couleurs, les informations centrés et des informations sous ligner.



*Figure 10 = Page Tkinter*

La page Tkinter présente correctement les informations de vol avec ICAO, le flight Number, la registration ainsi que l'image de l'avion concerné. En changeant le .dat ainsi que le .csv en cherchant un avion différent du précédent, les informations sur l'avion sont également correctes. Le programme est donc validé il récupère bien les informations de vols peut import le .dat et .csv utilisé.

## GNU radio :

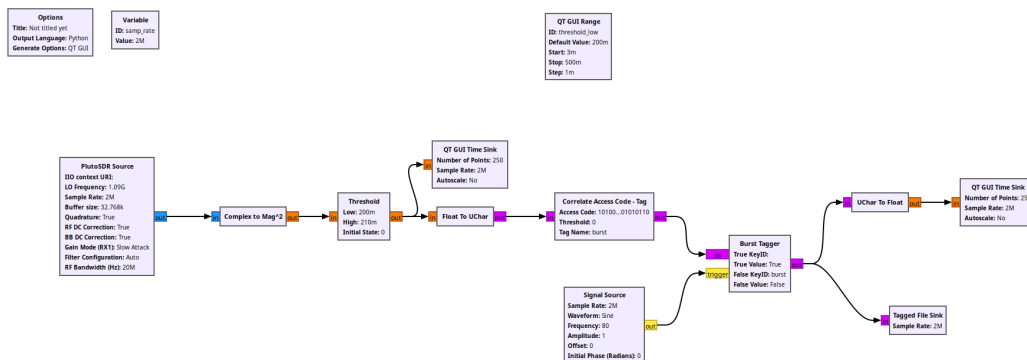
Le logiciel GNU radio va permettre de capturer les trames provenant des avions, pour cela la mise en place d'un programme est nécessaire pour la capture des trames au format .dat. Pour par la suite en extraire l'ICAO qui permet d'identifier la trame de l'avion intercepter. Cette expérience met en lumière l'utilisation des logiciels libres pour l'acquisition de signaux radio et le traitement des données.

Dans ce cadre du matériel est nécessaire pour intercepter ces trames, un récepteur SDR pour capturer les signaux ADS-B émis par l'avion ici un ADALM-PLUTO est utilisé avec une antenne spécialisée pour capter les fréquences ADS-B (1090 MHz).



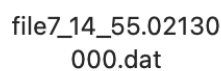
Figure 11 = Antenne et SDR

Ci-dessous le programme sur le logiciel GNU radio permettant d'intercepter les trames :



Ce programme GNU Radio est conçu pour capturer et traiter des signaux radio en particulier des trames ADS-B provenant d'un avion avec les matériels expliqués précédemment. Il commence par capter le signal via une source SDR via le bloc PlutoSDR Source qui reçoit les données de l'antenne. Ensuite la conversion de complex vers  $\text{Mag}^2$  qui est utilisée pour extraire et analyser la puissance d'un signal, puis un bloc de seuil (Threshold) est appliqué pour éliminer le bruit et ne conserver que les signaux pertinents. Les trames ADS-B sont extraites du signal à l'aide d'un bloc de corrélation, qui détecte et décode les trames selon un code spécifique. Enfin les informations extraites sont envoyées vers des blocs de visualisation (Data Sink, Tagged Stream), permettant de suivre et d'enregistrer les données en temps réel comme les trames des avions. Cette

Ci-dessus l'interception d'une trame qui va être par la suite enregistrer sous forme de .dat



The screenshot shows the Mousepad application window. The title bar reads "Mousepad". Below it, a message box states: "Le document n'est pas valide UTF-8. D'autres encodages partiellement valides ont été trouvés, veuillez choisir ci-dessous". There are two radio button options: "Par défaut (UTF-8, partiel)" which is unselected, and "Autre (partiel) : ANSIX3.4-1968" which is selected. A dropdown menu next to the second option also displays "ANSI\_X3.4-1968". Below the options, a horizontal row of small boxes contains various byte sequences, some highlighted in blue. At the bottom right, there are two buttons labeled "Annuler" and "Valider".

Figure 14 = trame intercepté



Voici la trame intercepter par GNU radio. Maintenant grâce au programme python le but va être d'extraire l'ICAO de l'avion et de vérifier sur flightRadar l'interception est correct.

The screenshot shows a Python IDE with a script named 'Registration.py' and its console output. The script processes a binary file 'file7\_14\_55\_02130000.dat' to extract an ICAO code. The console output shows the ICAO code '4D2250' and the flight number 'ZBXGN'.

```

1 import sys
2 import csv
3
4 # Chemin du fichier CSV contenant les immatriculations
5 BINARY_FILE_PATH = "file7_14_55_02130000.dat"
6
7
8 # Extraction dynamique de l'ICAO depuis un fichier binaire
9 with open(BINARY_FILE_PATH, 'rb') as binary_file:
10     binary_data = list(binary_file.read(107 * 2))
11
12 # Conversion des octets en une chaîne binaire
13 binary_message = "".join(str(binary_data[i * 2]) for i in range(107))
14
15 # Extraction et conversion de l'ICAO (bits 8 à 32)
16 icao_hex = format(int(binary_message[8:32], 2), '06x')
17
18 # Extraction et conversion du numéro de vol (bits 40 à 88)
19 flight_number = ""
20 for i in range(8): # Maximum 8 caractères
21     char_bits = binary_message[40 + i * 6: 46 + i * 6]
22     if len(char_bits) < 6:
23         break
24     char_value = int(char_bits, 2)
25     if 1 <= char_value <= 26:
26         flight_number += chr(char_value + 64) # Lettres A-Z
27     elif 48 <= char_value <= 57:
28         flight_number += chr(char_value) # Chiffres 0-9
29
30 # Affichage des résultats
31 print(f"ICAO: {icao_hex}")
32

```

Console Output:

```

File ~/Desktop/GNU/Enregistrement lucas/Registration.py:84
  print(f"ICAO: {icao_hex}")
  print(f"Flight Number: {flight_number.strip()}")
--> 84 print(f"Registration: {registration}")

NameError: name 'registration' is not defined

In [14]: Runfile 'Users/lucci/Desktop/GNU/Enregistrement lucas/Registration.py' --wdir
ICAO: 4D2250

In [15]: Runfile 'Users/lucci/Desktop/GNU/Enregistrement lucas/Registration.py' --wdir
ICAO: 4D2250

In [16]: Runfile 'Users/lucci/Desktop/GNU/Enregistrement lucas/Registration.py' --wdir
ICAO: 4D2250

In [17]: Runfile 'Users/lucci/Desktop/GNU/Enregistrement lucas/Registration.py' --wdir
ICAO: 4D2250

In [18]: Runfile 'Users/lucci/Desktop/GNU/Enregistrement lucas/Registration.py' --wdir
ICAO: 4D2250

In [19]:

```

Figure 14 = Python affichage ICAO

Le programme intègre juste de les parties de la conversion des octets en une chaîne de caractère, l'extraction et la conversion de l'icao qui est du bit 8 à 32. Permettant d'extraire correctement ICAO qui est dans ce cas 4D2250. FlightRadar va permettre de vérifier le résultat obtenu.

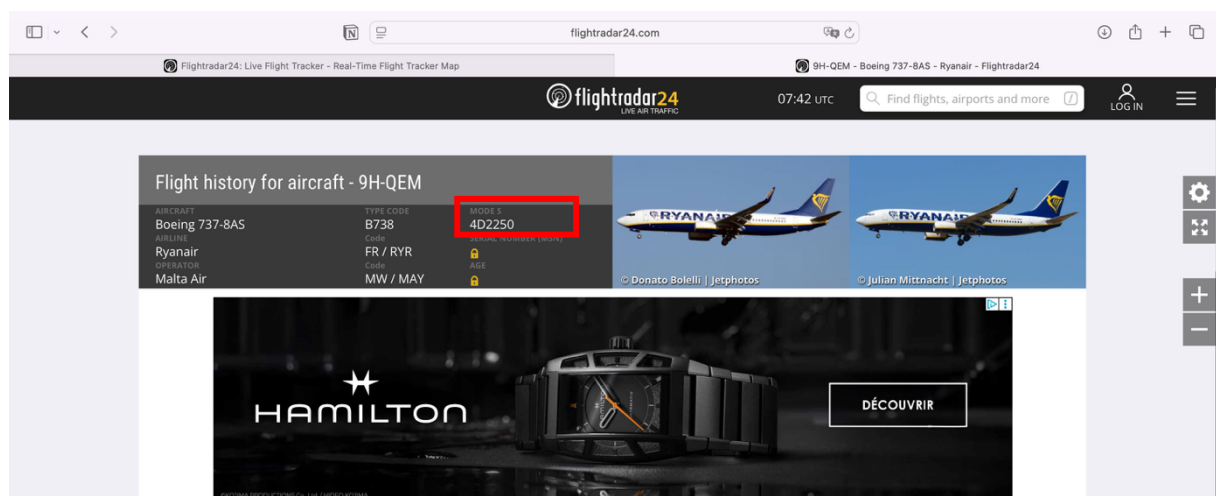


Figure 15 = ICAO sur flightradar



L'ICAO est encadré en rouge, la trame interceptor existe bien. Sur flightRadar, il est possible de vérifier le trajet que l'avion effectue.

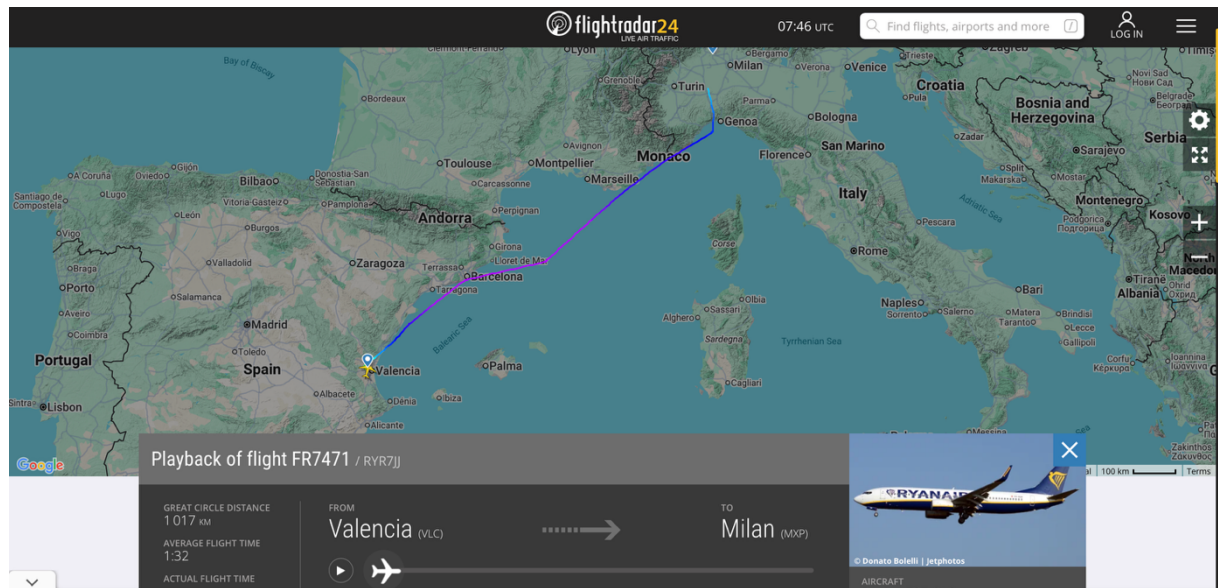


Figure 16 = flightradar vol

Le trajet est cohérent par rapport à l'interception, l'avion passe bien dans les alentours de la garde (d'où le signal a été intercepté). La partie GNU radio est donc validé de l'interception de la trame en .dat à l'analyse sur python jusqu'à la vérification sur FlightRadar.

Le matériel utilisé est un analyseur de réseau, une antenne ainsi qu'un kit de calibrage.



Figure 17 = Connecteurs

Chaque connecteur correspond un type en bleu le circuit ouvert, en rouge le court-circuit et en vert la charge adaptée.

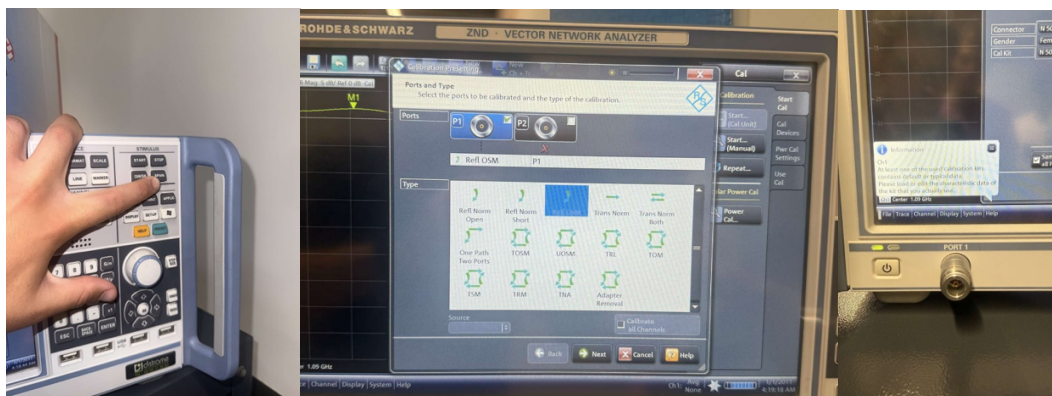
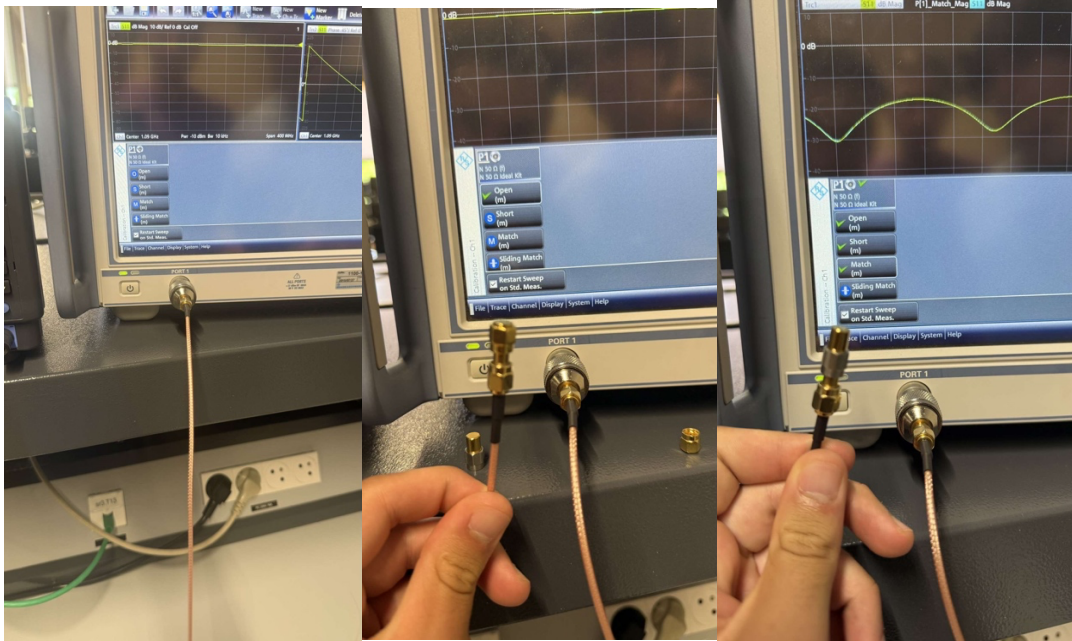


Figure 18 = Réglage de l'analyseur de réseau

Par la suite, le calibrage de la machine à l'aide des trois connecteurs présentes précédemment.



*Figure 19 = Calibrage de l'antenne avec les connecteurs*

Une fois le calibrage de la machine fait avec chaque connecteur afin de valider les étapes de l'analyseur de réseau. Il faut régler la bande passante à 400 Mhz ainsi que le center à 1090 MHz sur l'analyseur.



*Figure 20 = Réglage bande passante et center*

Ainsi que le ROSS pour qu'il soit inférieur à 2 et que RL soit inférieur à 10 dB.



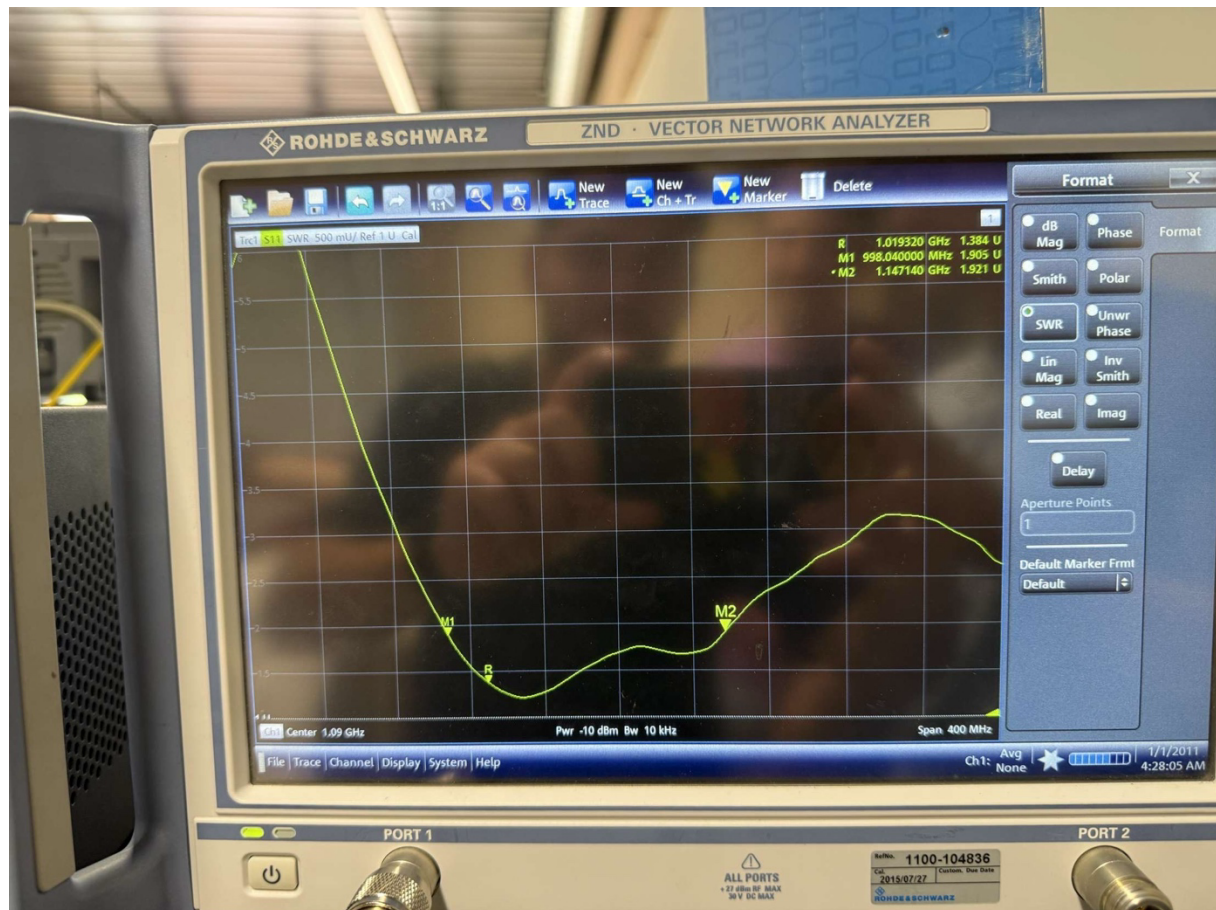


Figure 21 = Affichage SWR sur l'analyseur de réseau

A la suite de cela, il est possible d'afficher SWR qui est une mesure pour évaluer l'adaptation d'impédance entre une antenne et son alimentation. Le SWR mesure la qualité d'adaptation d'impédance d'un système, c'est-à-dire à quel point l'impédance de l'antenne et celle de la ligne de transmission sont en harmonie (généralement 50 ohms). Si l'impédance de l'antenne correspond exactement à celle de la ligne de transmission, alors toute l'énergie sera transférée efficacement, et il n'y aura pas de réflexion du signal.

La mesure obtenue indique un S11 de 500 mU avec un SWR calibré à 1 U. Ces résultats ont été obtenus après calibration de l'instrument, assurant ainsi la précision des mesures.

Un S11 faible indique une faible réflexion du signal, ce qui suggère une bonne adaptation de l'impédance entre l'antenne et la ligne de transmission. Par conséquent, un SWR proche de 1 confirme cette adaptation optimale, permettant une transmission efficace du signal avec peu de pertes.

Ces résultats indiquent que l'antenne est correctement adaptée, garantissant une faible réflexion et une transmission optimale de l'énergie.

Par la suite, un test équivalent a été fait sur un NanoVNA-F qui est un analyseur de réseau vectoriel. Qui permet également de mesurer les caractéristiques de transmission et de réflexion des antennes. Le VNA est particulièrement utilisé pour caractériser des réseaux linéaires en termes de paramètres S (paramètres de diffusion, comme S11, S21).



Figure 22 = NanoVNA-F

Le NanoVNA-F a besoin également d'un calibrage dans le même principe que le précédent avec les différents connecteurs.

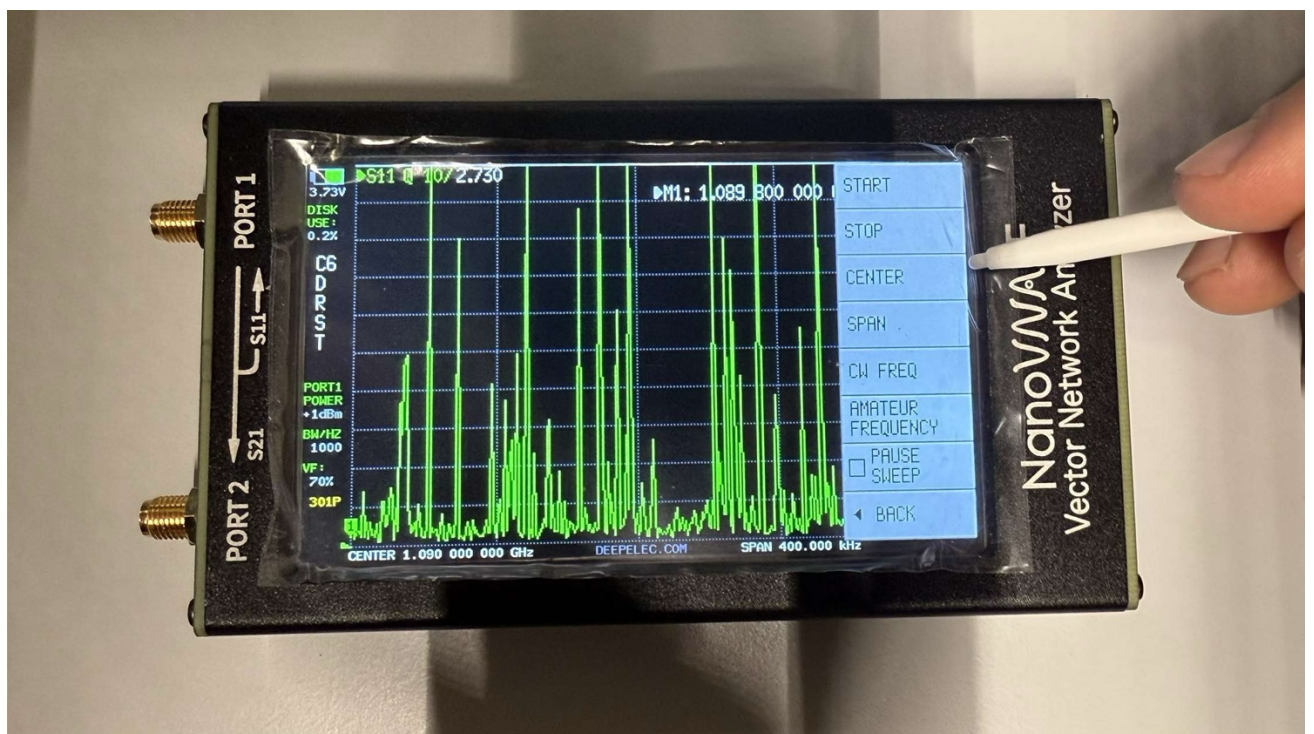


Figure 23 = Réglage NanoVNA-F

Dans le même principe, les différents réglages sont effectués comme le center et le span.

Grâce à cet appareil il est également possible de mesurer précisément le SWR. Le calibrage effectué sur le NanoVNA-F assure la précision des mesures, en éliminant les erreurs liées aux câbles connecteurs et autres imperfections du système de test.

En conclusion, l'utilisation du NanoVNA-F permet de s'assurer que l'antenne est correctement adaptée à la ligne de transmission assurant ainsi une transmission optimale du signal et minimisant les pertes dues à une mauvaise adaptation. Cela est essentiel pour garantir des performances efficaces et fiables.

En résumé, bien que le NanoVNA-F ne soit pas aussi précis ni aussi complet que l'analyseur de réseau vu précédemment, mais il représente une solution abordable et suffisamment précise pour des tests d'adaptation d'impédance et des mesures de réflexion dans de nombreuses applications courantes tout en étant une option largement plus économique.

## Conclusion :

En conclusion ce projet a permis d'implémenter et de tester plusieurs technologies et outils pour la surveillance et l'analyse des données ADS-B. Les résultats ont montré l'efficacité du système dans la capture des trames et l'extraction des informations telles que l'ICAO et le Flight Number ainsi que leur vérification à l'aide de FlightRadar. L'utilisation de GNU Radio pour capter les signaux ADS-B et du programme Python pour analyser les données a donné des résultats satisfaisants. Les tests d'adaptation d'impédance réalisés avec des analyseurs de réseau ont confirmé que l'antenne était correctement calibrée, assurant ainsi une transmission optimale du signal. Ce projet démontre la fiabilité des technologies utilisées et prouve qu'il est possible d'obtenir des résultats précis tout en utilisant des équipements accessibles comme le NanoVNA-F.